CS301 Term Project

2025–2026 Spring

Project Report

Group 26

Group Members:

Abdul Momin Alam - 33622
Abdallah Al Homsi - 34565
Rand Mo Khaled - 35003

Contents

1 Problem Description

Table 1: Description of the Subset Sum Problem

Name	Subset Sum
Input	A finite list of positive integers $S = \{a_1, a_2, \ldots, a_n\}$ and a target integer T.
Question	Does there exist a subset $S' \subseteq S$ such that the sum of the elements in S' is exactly equal to T?

1.1 Overview

The Subset Sum problem is a classic problem in computer science and algorithm design. In this problem, we are given a finite set or list of integers and a target value. The main question is whether some subset of the given numbers can be selected so that their sum is exactly equal to the target value.

For example, if the input set is $\{3, 5, 7, 10\}$ and the target value is 15, then the answer is yes because the subset $\{5, 10\}$ adds up to 15. However, if the target value was 14, then there may be no subset from this set that adds up exactly to 14.

The problem may look simple for small inputs, but it becomes much harder as the number of elements increases. This is because every element can either be included or not included in a subset. Therefore, for n elements, there are 2^n possible subsets. As a result, checking all possible subsets becomes inefficient for large inputs.

The Subset Sum problem is important because it represents the difficulty of many real-world decision and optimization problems where a selection must be made from a larger group of options while satisfying a numerical constraint.

1.2 Decision Problem

The decision version of the Subset Sum problem asks a yes-or-no question.

Given a set of integers $S = \{a_1, a_2, \ldots, a_n\}$ and a target integer T, the question is:

Does there exist a subset of S whose elements sum exactly to T?

Formally, the decision problem can be written as:

$$\text{Given } S = \{a_1, a_2, \ldots, a_n\} \text{ and target } T,$$

determine whether there exists a subset $S' \subseteq S$ such that:

$$\sum_{a_i \in S'} a_i = T$$

If such a subset exists, the answer is "yes." If no such subset exists, the answer is "no."

For example, if $S = \{2, 4, 6, 9\}$ and $T = 10$, the answer is yes because the subset $\{4, 6\}$ has a sum of 10.

1.3 Optimization Problem

The optimization version of the Subset Sum problem is slightly different from the decision version. Instead of only asking whether an exact subset exists, the goal is to find the best possible subset according to an objective.

A common optimization version is:

> Given a set of positive integers $S = \{a_1, a_2, \ldots, a_n\}$ and a target value T, find a subset $S' \subseteq S$ such that the sum of the selected elements is as large as possible without exceeding T.

Formally, the objective is to maximize:

$$\sum_{a_i \in S'} a_i$$

subject to:

$$\sum_{a_i \in S'} a_i \leq T$$

This means the algorithm should choose a subset whose total value is closest to the target without going over it.

For example, if $S = \{4, 8, 10, 15\}$ and $T = 18$, the subset $\{8, 10\}$ has a sum of 18, so it is optimal because it reaches the target exactly. If the target was 19, then $\{4, 15\}$ gives 19, which is also optimal. If the target was 17, then $\{4, 10\}$ gives 14, while $\{15\}$ gives 15, so $\{15\}$ would be better because it is closer to 17 without exceeding it.

1.4 Example Illustration

The Subset Sum problem can be understood more clearly through different examples.

Example 1: A subset exists

Let:

$$S = \{3, 5, 7, 10\}, \quad T = 15$$

The subset $\{5, 10\}$ has a sum of:

$$5 + 10 = 15$$

Therefore, the answer to the decision problem is yes.

Example 2: No exact subset exists

Let:

$$S = \{2, 6, 8, 13\}, \quad T = 11$$

Possible subset sums include values such as 2, 6, 8, 13, $2 + 6 = 8$, $2 + 8 = 10$, and $6 + 8 = 14$. None of the possible subsets sum exactly to 11. Therefore, the answer to the decision problem is no.

Example 3: Multiple valid subsets exist

Let:

$$S = \{1, 3, 4, 5, 6\}, \quad T = 9$$

There are multiple subsets that sum to 9, such as:

$$\{3, 6\}, \quad \{4, 5\}, \quad \{1, 3, 5\}$$

In this case, the decision problem only needs to return yes because at least one valid subset exists. However, if we are analyzing different algorithms, we may also compare which subset each algorithm returns.

Example 4: Optimization version

Let:

$$S = \{6, 9, 11, 13\}, \quad T = 20$$

The subset $\{9, 11\}$ gives:

$$9 + 11 = 20$$

So this is an optimal solution because it reaches the target exactly.

Now consider:

$$S = \{6, 9, 13\}, \quad T = 20$$

The subset $\{6, 13\}$ gives 19, while $\{9\}$ gives 9, $\{13\}$ gives 13, and $\{6, 9\}$ gives 15. Since 19 is the largest sum that does not exceed 20, $\{6, 13\}$ is the optimal solution for the optimization version.

1.5 Real World Applications

The Subset Sum problem has several real-world applications because many practical problems involve selecting a group of items that must satisfy a certain numerical limit or target.

One application is budgeting. A person or organization may have a fixed budget and a list of possible purchases or projects, each with a cost. The goal may be to choose a combination of items that uses as much of the budget as possible without going over the limit.

Another application is resource allocation. In business or operations, managers may need to assign limited resources to selected tasks. If each task requires a certain amount of resources, the problem becomes similar to choosing a subset that fits within a given resource capacity.

Subset Sum is also related to packing and loading problems. For example, when loading items into a truck, container, or storage space, each item has a weight or size, and there is a maximum capacity. The goal is to select items that fit within the capacity while using the space efficiently.

Another important application is cryptography. Some older cryptographic systems were based on the difficulty of solving Subset Sum-like problems. Although many of these systems are no longer commonly used, the problem remains important in the study of computational complexity and security.

The problem can also appear in financial decision-making. For example, an investor may want to select a combination of investments that reaches a target amount of money or risk level as closely as possible.

Overall, Subset Sum is useful because it models situations where a decision-maker must select some items from a larger set while satisfying a numerical goal or constraint.

1.6 Hardness of the Problem

The Subset Sum problem is a well-known computationally difficult problem. Its decision version is NP-complete, and its optimization version is NP-hard.

Theorem: The decision version of the Subset Sum problem is NP-complete.

Proof Explanation:

The Subset Sum problem belongs to NP because if a subset is given as a proposed solution, it can be verified in polynomial time by adding the selected elements and checking whether their sum equals the target value. This verification process takes at most linear time with respect to the number of elements.

The problem is also NP-hard. Subset Sum is one of the classical NP-complete problems shown in computational complexity literature. Therefore, since the decision version of Subset Sum is both in NP and NP-hard, it is NP-complete.

The optimization version of Subset Sum is NP-hard because solving the optimization version would also allow us to solve the decision version. If an algorithm could efficiently find the best subset sum that does not exceed the target, then we could check whether the best sum is exactly equal to the target. Therefore, the optimization version is at least as hard as the decision version.

This hardness result is commonly cited from Karp's work on reducibility among combinatorial problems and from Garey and Johnson's book on NP-completeness [1, 2].

2 Algorithm Description

2.1 Brute Force Algorithm

2.1.1 Overview

The brute force algorithm for the Subset Sum problem is an exact algorithm that checks every possible subset of the given input set. Since each element can either be included or not included in a subset, an input set with n elements has 2^n possible subsets. The brute force method examines these subsets and calculates their sums.

In this project, the brute force algorithm is used for the optimization version of Subset Sum. The goal is to find a subset whose sum is as large as possible without exceeding the target value T. If the algorithm finds a subset whose sum is exactly equal to T, then this also answers the decision version of the problem with "yes." If no subset adds exactly to T, then the algorithm still returns the best possible subset that does not exceed the target.

The brute force algorithm is guaranteed to find the optimal solution because it checks all possible subsets. However, it is not efficient for large inputs because the number of possible subsets grows exponentially as the number of elements increases. Therefore, this algorithm is useful for correctness comparison and quality testing, but it is not practical for very large input sizes.

This algorithm does not use a complex design technique such as dynamic programming or divide-and-conquer. Instead, it uses exhaustive search. Exhaustive search means that every possible candidate solution is generated and evaluated.

2.1.2 Pseudocode

The brute force algorithm considers every possible subset and keeps track of the subset with the largest sum that does not exceed the target.

Algorithm 1: Brute Force Subset Sum

Input: A list of positive integers $S = \{a_1, a_2, \ldots, a_n\}$, target value T

Output: Best subset found, best subset sum, and decision answer

1. $best\_subset \leftarrow$ empty set
2. $best\_sum \leftarrow 0$
3. for each possible subset X of S do
4. $\quad current\_sum \leftarrow$ sum of all elements in X
5. $\quad$ if $current\_sum \leq T$ and $current\_sum > best\_sum$ then

6. $best\_subset \leftarrow X$

7. $best\_sum \leftarrow current\_sum$

8. end if

9. end for

10. if $best\_sum = T$ then

11. $decision\_answer \leftarrow$ "yes"

12. else

13. $decision\_answer \leftarrow$ "no"

14. end if

15. return $best\_subset$, $best\_sum$, $decision\_answer$

The algorithm starts with an empty best subset and a best sum of zero. Then, it generates each possible subset of the input list. For each subset, it calculates the total sum. If the subset sum does not exceed the target and is greater than the current best sum, the algorithm updates the best subset and best sum.

At the end, if the best sum is exactly equal to the target value, the decision version of the problem has answer "yes." Otherwise, the answer is "no." However, even when the answer is "no," the algorithm still returns the closest possible subset sum that does not exceed the target.

2.2 Heuristic Algorithm

2.2.1 Overview

The heuristic algorithm used in this project is a greedy algorithm. A heuristic algorithm is a practical algorithm that tries to find a good solution efficiently, but it does not always guarantee the optimal solution. This is useful for the Subset Sum problem because the exact brute force algorithm becomes too slow when the input size becomes large.

The greedy heuristic works by first sorting the numbers in descending order. Then, it goes through the sorted list one by one. If adding the current number does not make the total sum exceed the target value T, the number is added to the selected subset. If adding the number would make the total exceed the target, the number is skipped. The algorithm continues until all numbers have been considered.

For example, if the input set is $\{8, 6, 9, 5\}$ and the target is 14, the algorithm first sorts the numbers as $\{9, 8, 6, 5\}$. It selects 9 because $9 \leq 14$. It skips 8 because $9 + 8 = 17$, which

exceeds 14. It then skips 6 because $9 + 6 = 15$, which also exceeds 14. Finally, it selects 5 because $9 + 5 = 14$. The final selected subset is $\{9, 5\}$, with a sum of 14.

This greedy approach is efficient because it only sorts the list and then scans through it once. However, it may fail to find the optimal answer in some cases. For example, if $S = \{10, 9, 6, 5\}$ and $T = 11$, the greedy algorithm selects 10 first and then cannot add anything else, giving a sum of 10. However, the optimal solution is $\{6, 5\}$, which gives a sum of 11. This shows that the greedy heuristic is fast but not always optimal.

This algorithm follows the greedy design technique because it makes a locally best choice at each step. Specifically, it tries to select the largest currently available number that can fit without exceeding the target. Once a number is selected or skipped, the algorithm does not go back and reconsider that decision.

Since this is used as a heuristic algorithm and not as a guaranteed approximation algorithm, we do not claim a fixed approximation ratio. Because no fixed worst-case approximation ratio is claimed for this greedy approach on general Subset Sum instances, it is treated as a heuristic and evaluated empirically rather than as an approximation algorithm with a proven ratio bound.

2.2.2 Pseudocode

The greedy heuristic algorithm sorts the input numbers from largest to smallest and then adds numbers when possible without exceeding the target.

Algorithm 2: Greedy Heuristic Subset Sum

Input: A list of positive integers $S = \{a_1, a_2, \ldots, a_n\}$, target value T

Output: Selected subset, selected subset sum, and $exact\_match\_found$

1. Sort S in descending order
2. $selected\_subset \leftarrow$ empty set
3. $current\_sum \leftarrow 0$
4. for each element x in sorted S do
5. if $current\_sum + x \leq T$ then
6. add x to $selected\_subset$
7. $current\_sum \leftarrow current\_sum + x$
8. else
9. skip x

10. end if

11. end for

12. if $current\_sum = T$ then

13. $exact\_match\_found \leftarrow True$

14. else

15. $exact\_match\_found \leftarrow False$

16. end if

17. return $selected\_subset$, $current\_sum$, $exact\_match\_found$

The algorithm begins by sorting the input list in descending order. It then initializes an empty selected subset and sets the current sum to zero. For each number in the sorted list, the algorithm checks whether adding that number would keep the total sum less than or equal to the target. If yes, the number is added. If no, the number is skipped.

At the end, the algorithm returns the selected subset and its sum. If the selected subset sum is exactly equal to the target, then the heuristic found an exact match. However, if the selected subset sum is not equal to the target, this does not prove that no exact subset exists. It only means that the greedy heuristic did not find one. Unlike the brute force algorithm, this greedy algorithm does not guarantee that the selected subset is optimal. It only guarantees that the returned subset is valid because its sum never exceeds the target.

3 Algorithm Analysis

3.1 Brute Force Algorithm

3.1.1 Correctness Analysis

The brute force algorithm is an exact algorithm for the optimization version of the Subset Sum problem. Its goal is to find a subset whose sum is as large as possible without exceeding the target value T.

Theorem: The brute force algorithm returns an optimal subset whose sum is the largest possible value less than or equal to the target T.

Proof: The brute force algorithm considers every possible subset of the input set S. Since each element can either be included or excluded, checking all possible subsets means that no candidate solution is ignored.

For each subset, the algorithm calculates the subset sum. If the sum is less than or equal to the target value T, then the subset is feasible. Among all feasible subsets, the algorithm keeps track of the one with the largest sum found so far.

Because the algorithm checks every possible subset, it must eventually check the optimal subset as well. When it checks the optimal subset, the best sum is updated if that subset has a larger feasible sum than all previous subsets. Therefore, by the end of the algorithm, the stored subset must be the best feasible subset.

If the best sum is exactly equal to T, then the decision version of Subset Sum has the answer "yes." If the best sum is not equal to T, then no subset sums exactly to T, so the decision version has the answer "no." This is correct because every possible subset has been checked.

Therefore, the brute force algorithm correctly returns the optimal subset for the optimization version and the correct yes/no answer for the decision version. □

3.1.2 Time Complexity Analysis

Let n be the number of elements in the input set S.

Each element has two choices: it can either be included in a subset or excluded from a subset. Therefore, the total number of possible subsets is:

$$2^n$$

The brute force algorithm checks every possible subset. For each subset, calculating the sum can take up to $O(n)$ time because the algorithm may need to examine each element in the subset.

Therefore, the total worst-case time complexity is:

$$O(n \cdot 2^n)$$

This bound is also tight for the described brute force method because the algorithm must examine all 2^n subsets and may spend up to linear time evaluating each subset. Therefore, the time complexity can be written as:

$$\Theta(n \cdot 2^n)$$

This shows that the brute force algorithm has exponential time complexity. It is practical for small input sizes, but it becomes inefficient very quickly as n increases.

3.1.3 Space Complexity Analysis

The brute force algorithm stores the input list, the current subset being evaluated, and the best subset found so far.

If the input list is not counted as extra space, then the main additional space comes from storing the current subset, the best subset, and a few variables such as $current\_sum$ and $best\_sum$.

In the worst case, both the current subset and the best subset may contain up to n elements. Therefore, the auxiliary space complexity is:

$$O(n)$$

If the input list is included, the total space used is also linear in the number of elements. Therefore, the space complexity is:

$$O(n)$$

This means that although the brute force algorithm is very slow in terms of time, it does not require exponential memory as long as it generates subsets one at a time instead of storing all subsets at once.

3.2 Heuristic Algorithm

3.2.1 Correctness Analysis

The heuristic algorithm used in this project is a greedy algorithm. It sorts the numbers in descending order and adds each number to the selected subset only if doing so does not make the current sum exceed the target value T.

Since this is a heuristic algorithm, it is not expected to always return the optimal subset. For this project, correctness means that the algorithm returns a valid feasible subset. A

valid subset is one whose sum does not exceed the target value T.

Theorem: The greedy heuristic algorithm always returns a valid subset whose sum is less than or equal to the target value T.

Proof: The algorithm starts with an empty subset and a current sum of 0. Since $0 \leq T$, the initial subset is valid.

Then, the algorithm considers the elements one by one after sorting them in descending order. For each element x, the algorithm checks whether adding x to the current subset would keep the total sum less than or equal to the target value T.

If:

$$current\_sum + x \leq T$$

then the algorithm adds x to the selected subset and updates the current sum.

If:

$$current\_sum + x > T$$

then the algorithm skips x.

Therefore, the algorithm only adds an element when the resulting sum remains feasible. At no point does the algorithm allow the selected subset sum to exceed the target value.

By the end of the algorithm, every selected element belongs to the original input set, and the total selected sum is less than or equal to T. Therefore, the algorithm always returns a valid subset.

However, the algorithm does not always return the optimal subset. This is because it makes greedy local choices and does not reconsider previous decisions. For example, selecting a large number early may prevent the algorithm from selecting a better combination of smaller numbers later. Therefore, the heuristic is correct only in the sense that it returns a valid feasible result, not necessarily an optimal one. □

3.2.2 Time Complexity Analysis

Let n be the number of elements in the input set S.

The greedy heuristic algorithm has two main steps.

First, it sorts the input list in descending order. Sorting n elements takes:

$$O(n \log n)$$

Second, the algorithm scans through the sorted list once. For each element, it checks whether the element can be added without exceeding the target. This scan takes:

$$O(n)$$

Therefore, the total worst-case time complexity is:

$$O(n \log n + n)$$

Since $n \log n$ grows faster than n, the simplified time complexity is:

$$O(n \log n)$$

This is also a tight bound because sorting dominates the running time. Therefore, the time complexity can be written as:

$$\Theta(n \log n)$$

This shows that the greedy heuristic is much more efficient than the brute force algorithm. It can handle much larger input sizes because its running time grows polynomially instead of exponentially.

3.2.3 Space Complexity Analysis

The greedy heuristic algorithm stores the selected subset and a few variables such as $current\_sum$.

If the sorting is done in place, then the sorting step does not require a full extra copy of the input list. However, the selected subset may contain up to n elements in the worst case. Therefore, the auxiliary space complexity is:

$$O(n)$$

If the algorithm creates a separate sorted copy of the input list, then this also requires $O(n)$ additional space. The selected subset also requires at most $O(n)$ space.

Therefore, in either case, the overall space complexity is:

$$O(n)$$

This means the greedy heuristic is efficient in both time and space compared to brute force. It does not require checking or storing all possible subsets.

4 Sample Generation

4.1 Explanation

To test and compare the algorithms, we need to generate random instances of the Subset Sum problem. A random instance of Subset Sum consists of a list of positive integers and a target value. The list represents the input list S, and the target value T represents the sum that the algorithm tries to reach exactly or approach without exceeding.

The sample generator used in this project is parametric, meaning that the user can control the size and structure of the generated problem instances. The main parameters are:

- n: the number of elements in the input list
- min_value: the smallest possible value of an element
- max_value: the largest possible value of an element
- $target\_ratio$: the percentage of the total sum used to create the target value
- $seed$: an optional value used to make the random generation reproducible

The generator first creates a list S containing n random positive integers between min_value and max_value. Then, it calculates the total sum of all elements in the list. The target value T is generated by multiplying the total sum by the $target\_ratio$ and taking the floor of the result.

For example, if the generated list is:

$$S = \{4, 10, 7, 15, 9\}$$

then the total sum is:

$$4 + 10 + 7 + 15 + 9 = 45$$

If $target\_ratio = 0.5$, then the target value is approximately:

$$T = 45 \times 0.5 = 22.5$$

Since the target must be an integer, it can be rounded down to:

$$T = 22$$

This approach is useful because it allows us to generate different types of test cases. If the $target\_ratio$ is small, the target will be much smaller than the total sum, so the algorithms

must choose only a few elements. If the *target_ratio* is large, the target will be closer to the total sum, so more elements may be selected.

The generated samples will be used later for initial testing, performance testing, quality testing, and functional testing. This allows the algorithms to be tested on many different input sizes instead of relying only on manually created examples.

4.2 Pseudocode

Algorithm 3: Random Subset Sum Instance Generator

Input: n, min_value, max_value, $target\_ratio$, optional seed

Output: A random list S and target value T

1. If seed is given, set the random seed.
2. $S \leftarrow$ empty list
3. For $i \leftarrow 1$ to n do:
4. Generate a random integer x between min_value and max_value.
5. Add x to S.
6. End for
7. $total\_sum \leftarrow$ sum of all elements in S
8. $T \leftarrow \lfloor total\_sum \times target\_ratio \rfloor$
9. Return S, T

The algorithm begins by setting the random seed if one is provided. This makes the output reproducible, meaning the same input parameters will generate the same sample. Then, it creates an empty list and generates n random positive integers within the given range. These values are stored in the list S.

After the list is created, the algorithm computes the total sum of the list. The target value is then calculated using the *target_ratio* parameter. Finally, the algorithm returns both the generated list and the target value.

4.3 Example Generated Instance

Suppose the generator is called with the following parameters:

$$n = 6$$

$$min\_value = 1$$

$$max\_value = 20$$

$$target\_ratio = 0.5$$

$$seed = 42$$

Using these parameters, one generated instance is:

$$S = \{4, 1, 9, 8, 8, 5\}$$

The total sum is:

$$4 + 1 + 9 + 8 + 8 + 5 = 35$$

The target value is:

$$T = \lfloor 35 \times 0.5 \rfloor = 17$$

Therefore, the generated Subset Sum instance is:

$$S = \{4, 1, 9, 8, 8, 5\}, \quad T = 17$$

For this instance, the algorithms will try to find a subset of S whose sum is exactly 17 for the decision version, or as close as possible to 17 without exceeding it for the optimization version.

5 Algorithm Implementation

5.1 Brute Force Python Code

The brute force algorithm was implemented in Python. This algorithm solves the optimization version of the Subset Sum problem by checking every possible subset of the input list. For each subset, it calculates the subset sum and checks whether the sum is less than or equal to the target value T.

The algorithm keeps track of the best subset found so far. The best subset is the subset with the largest sum that does not exceed the target. If the algorithm finds a subset whose sum is exactly equal to the target, then an exact match has been found. Since the brute force algorithm checks all possible subsets, it is guaranteed to find the optimal solution.

The implementation also records the running time of the algorithm using Python's `time.perf_counter()` function. This allows the running time of the brute force algorithm to be compared with the running time of the heuristic algorithm.

```
import itertools
import time

def brute_force_subset_sum(S, T):
    """
    Exact brute force algorithm for the optimization version of Subset Sum.
    It checks every possible subset and returns the subset with the largest sum
    that does not exceed the target T.
    """

    start_time = time.perf_counter()

    best_subset = []
    best_sum = 0
    n = len(S)

    for r in range(n + 1):
        for subset in itertools.combinations(S, r):
            current_sum = sum(subset)

            if current_sum <= T and current_sum > best_sum:
                best_subset = list(subset)
                best_sum = current_sum

            if best_sum == T:
                end_time = time.perf_counter()
```

```
                elapsed_time = end_time - start_time
                return best_subset, best_sum, True, elapsed_time

    end_time = time.perf_counter()
    elapsed_time = end_time - start_time

    exact_match_found = (best_sum == T)

    return best_subset, best_sum, exact_match_found, elapsed_time
```

5.2 Heuristic Python Code

The heuristic algorithm was also implemented in Python. The heuristic used in this project is a greedy algorithm. It first sorts the input list in descending order. Then, it scans the sorted list from largest to smallest. Each element is added to the selected subset only if adding it does not make the current sum exceed the target value T.

This algorithm is much faster than brute force because it does not check all possible subsets. Instead, it makes a sequence of local choices. However, the algorithm is not guaranteed to find the optimal subset. It may miss a better combination of smaller numbers because it selects larger numbers first.

The implementation returns the selected subset, the sum of the selected subset, whether the heuristic reached the target exactly, and the running time. The phrase "exact match found" only means that the heuristic found a subset equal to the target. If the heuristic does not find an exact match, this does not prove that no exact subset exists.

```
def greedy_subset_sum(S, T):
    """
    Greedy heuristic algorithm for the optimization version of Subset Sum.
    It sorts the values in descending order and adds an element if it does not
    make the current sum exceed the target T.
    """

    start_time = time.perf_counter()

    sorted_S = sorted(S, reverse=True)

    selected_subset = []
    current_sum = 0

    for x in sorted_S:
        if current_sum + x <= T:
```

```
            selected_subset.append(x)
            current_sum += x

    end_time = time.perf_counter()
    elapsed_time = end_time - start_time

    exact_match_found = (current_sum == T)

    return selected_subset, current_sum, exact_match_found, elapsed_time
```

5.3 Initial Testing Results

The following testing procedure was used for the initial implementation test:

1. Generate 20 random Subset Sum instances using the sample generator.
2. Run the brute force algorithm on each instance.
3. Run the greedy heuristic algorithm on the same instance.
4. Check whether each returned subset is valid.
5. Record the returned subset, subset sum, exact match result, and running time.

The purpose of this initial testing was to confirm that both implementations run correctly and return feasible outputs. Larger performance and quality experiments will be presented in later sections.

A table of the initial test results was generated in Python and saved as:

`section5_initial_testing_results.csv`

This file includes the input list, target value, brute force result, heuristic result, validity status, and running time for each of the 20 generated test cases.

Table 2: Summary of Initial Implementation Testing

Item	**Result**
Number of random instances	20
Input size range	$n = 5$ to $n = 14$
Brute force validity failures	0
Greedy validity failures	0
Brute force output type	Optimal feasible subset sum
Greedy output type	Valid feasible subset sum
Result file	`section5_initial_testing_results.csv`

No implementation failures were observed during the initial tests. In all 20 test cases, the brute force algorithm returned a valid subset whose sum was less than or equal to the target value. The greedy heuristic also returned a valid subset whose sum was less than or equal to the target value. As expected, the heuristic was faster than the brute force algorithm, but in some cases its result may be lower than the brute force result because the greedy method does not always find the optimal subset.

6 Performance Testing

6.1 Purpose of the Performance Test

The purpose of this section is to experimentally analyze the running time performance of the heuristic algorithm. The theoretical analysis in Section 3 showed that the greedy heuristic has time complexity $\Theta(n \log n)$, mainly because the input list is sorted before the greedy selection step. However, theoretical complexity describes worst-case growth, so it is also important to test how the algorithm performs in practice.

In this experiment, only the greedy heuristic algorithm was tested for large input sizes. The brute force algorithm was not used for large input sizes because its running time grows exponentially and becomes impractical as n increases. The heuristic algorithm, on the other hand, is expected to scale much more easily and can be tested on much larger instances.

6.2 Experimental Setup

The random instance generator from Section 4 was used to create Subset Sum instances of different sizes. For each input size, the generator created a list of positive integers and a target value. The target value was set as a percentage of the total sum of the generated list.

The input sizes used in this experiment were:

$$n = 50, 100, 200, 400, 800, 1600, 3200, 6400, 10000$$

For each input size, the greedy heuristic was executed multiple times. The running time was measured using Python's `time.perf_counter()` function. For smaller input sizes, the algorithm runs very quickly, so repeated runs were grouped together in batches to obtain more stable timing measurements.

For each input size, the following values were recorded:

- mean running time
- standard deviation
- standard error
- t-value
- number of runs
- 90% confidence interval
- margin of error b
- ratio b/a, where a is the mean running time

The experiment used a 90% confidence level. The confidence interval was written in the form:

$$[a - b, a + b]$$

where a is the sample mean and b is the margin of error.

Following the lecture method, the standard error was calculated as:

$$s_m = \frac{sd}{\sqrt{N}}$$

where sd is the sample standard deviation and N is the number of timing measurements.

The margin of error was calculated as:

$$b = t \times s_m$$

where t is the t-value for a two-sided 90% confidence interval.

The confidence interval was considered narrow enough when:

$$\frac{b}{a} < 0.1$$

This follows the project requirement that the confidence interval should be narrow enough for each input size.

6.3 Performance Testing Results

The performance test results were generated using Python and saved as:

`section6_performance_results.csv`

The table includes the input size, number of runs, mean running time, standard deviation, standard error, t-value, 90% confidence interval, margin of error, and b/a value.

Table 3: Performance testing results for the greedy heuristic algorithm

n	Runs	Mean Time	Std Dev	Std Error	t-value	90% CI Lower	90% CI Upper	b	b/a
50	92	0.00001103	0.00000635	0.00000066	1.6618	0.00000993	0.00001213	0.00000110	0.0998
100	87	0.00003092	0.00001732	0.00000186	1.6628	0.00002783	0.00003400	0.00000309	0.0999
200	146	0.00005862	0.00004271	0.00000354	1.6554	0.00005277	0.00006447	0.00000585	0.0998
400	53	0.00015374	0.00006643	0.00000913	1.6747	0.00013846	0.00016902	0.00001528	0.0994
800	85	0.00035591	0.00019563	0.00002122	1.6632	0.00032062	0.00039120	0.00003529	0.0992
1600	59	0.00080014	0.00036299	0.00004726	1.6716	0.00072115	0.00087914	0.00007899	0.0987
3200	49	0.00135071	0.00056080	0.00008011	1.6772	0.00121634	0.00148508	0.00013437	0.0995
6400	59	0.00378894	0.00172237	0.00022423	1.6716	0.00341412	0.00416376	0.00037482	0.0989
10000	60	0.00576289	0.00266839	0.00034449	1.6711	0.00518722	0.00633856	0.00057567	0.0999

Across the tested input sizes, the greedy heuristic algorithm remained efficient. Even for the largest input size, $n = 10000$, the mean running time was only about 0.00576289

seconds. This shows that the heuristic scales well in practice because it does not examine all possible subsets. Instead, it sorts the input list and then scans through it once.

The b/a values were checked for every input size. All b/a values were below 0.1, which means that the confidence intervals were narrow enough according to the project requirement. The largest b/a value was 0.0999, which is still below the required threshold. Therefore, the timing results satisfy the 90% confidence interval requirement.

6.4 Visualization of Running Time

The first chart shows the mean running time of the greedy heuristic as the input size increases.

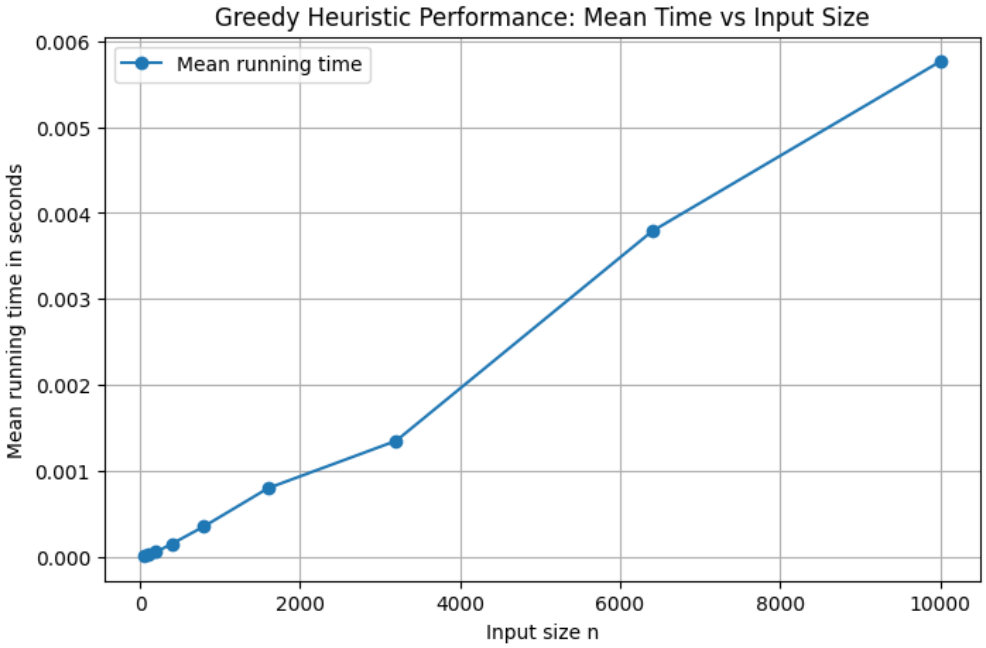

Figure 1: Mean running time of greedy heuristic for different input sizes

The graph shows that the running time increases as the input size increases. This is expected because larger lists require more time to sort and scan. However, the increase is much slower than what would be expected from an exponential-time algorithm. This supports the theoretical result that the heuristic algorithm scales efficiently.

6.5 Fitted Line and Running Time Expression

Since the theoretical time complexity of the greedy heuristic is $\Theta(n \log n)$, a fitted line was created using $n \log_2(n)$ as the independent variable and the mean running time as the dependent variable.

The fitted model has the following form:

$$Time \approx c_1 \times n \log_2(n) + c_2$$

Using the updated timing measurements, the fitted model is approximately:

$$Time \approx 0.0000000440 \times n \log_2(n) - 0.0000096479$$

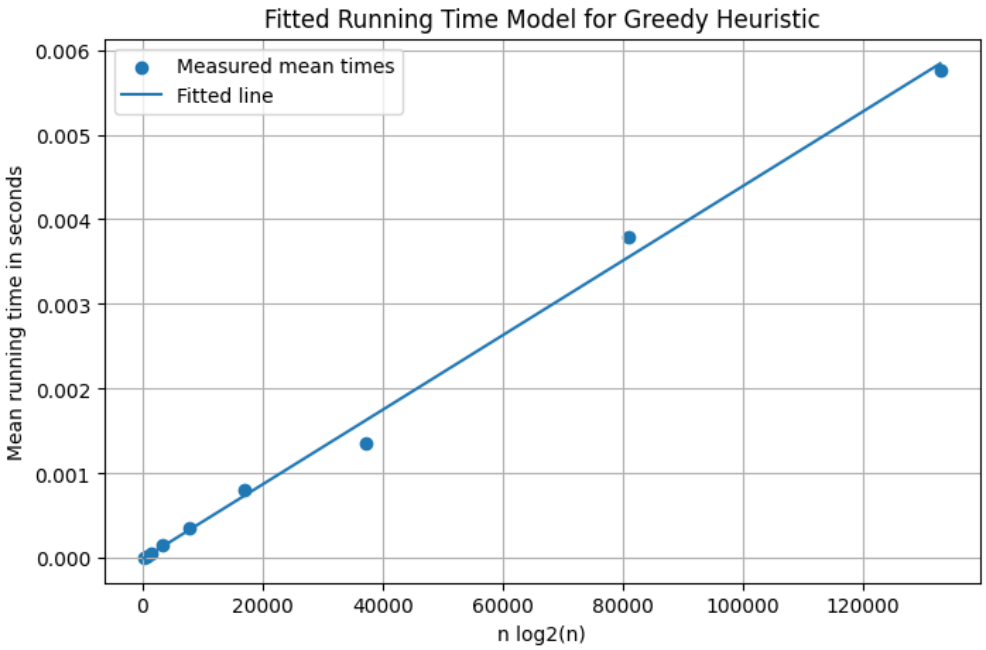

Figure 2: Fitted running time model for greedy heuristic

The fitted line helps show whether the practical running time follows the expected theoretical growth pattern. Since the greedy heuristic sorts the list first, the expected practical behavior is close to $n \log n$. The fitted line helps derive a practical running time expression based on the experimental measurements.

6.6 Log-Log Plot

A log-log plot was also created to better visualize the growth trend of the running time over a wide range of input sizes.

Figure 3: Log-log plot of greedy heuristic running time

The log-log plot makes it easier to see how the running time grows as the input size becomes large. The pattern supports the conclusion that the greedy heuristic grows much more slowly than the brute force algorithm. This makes the heuristic suitable for large Subset Sum instances where brute force is not practical.

6.7 Performance Discussion

The experimental results are consistent with the theoretical analysis from Section 3. The greedy heuristic has theoretical time complexity $\Theta(n \log n)$, and the experiment shows that the running time increases gradually as the input size increases.

The main reason for this behavior is that the algorithm only performs two major operations. First, it sorts the input list. Second, it scans through the sorted list once and selects elements when possible. It does not generate all possible subsets, so it avoids the exponential growth seen in the brute force algorithm.

The confidence interval results also support the reliability of the timing measurements. Since repeated measurements were used for every input size, the reported mean running times are more reliable than single timing measurements. Since all b/a values are below 0.1, the intervals are narrow enough according to the project requirement.

Overall, the performance experiment shows that the greedy heuristic is efficient and scalable. It can handle much larger input sizes than the brute force algorithm. However, performance alone does not show whether the heuristic finds high-quality solutions. For this reason, the quality of the heuristic solutions is analyzed separately in Section 7.

7 Quality Testing

7.1 Purpose of the Quality Test

The purpose of this section is to experimentally analyze the solution quality of the greedy heuristic algorithm. The performance experiment in Section 6 showed that the greedy heuristic runs efficiently on large input sizes. However, speed alone is not enough to evaluate a heuristic algorithm. Since the heuristic does not always guarantee the optimal answer, we also need to measure how close its solutions are to the optimal solutions.

To evaluate solution quality, the greedy heuristic was compared against the brute force algorithm. The brute force algorithm checks every possible subset, so it gives the optimal subset sum for each test instance. The greedy heuristic result can then be compared to this optimal result.

The main quality measure used in this experiment is the quality ratio:

$$Quality\ Ratio = \frac{Greedy\ Sum}{Brute\ Force\ Optimal\ Sum}$$

A quality ratio close to 1 means that the greedy heuristic found a solution very close to the optimal solution. A quality ratio equal to 1 means the greedy heuristic found an optimal solution. A lower ratio means the greedy solution was farther away from the brute force optimal result.

7.2 Experimental Setup

The random instance generator from Section 4 was used to create test instances for the quality experiment. Unlike the performance experiment, this quality test used smaller input sizes because the brute force algorithm must be run to find the true optimal answer. Since brute force has exponential time complexity, it becomes impractical for large values of n.

The input sizes used in this experiment were:

$$n = 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18$$

For each input size, 30 random instances were generated. Each instance was solved using both the brute force algorithm and the greedy heuristic algorithm.

For each test case, the following values were recorded:

- input size n
- target value T
- brute force optimal sum

- greedy heuristic sum
- quality gap
- quality ratio
- whether the greedy result matched the optimal result
- whether the greedy result was valid

The quality gap was calculated as:

$$Quality\ Gap = Brute\ Force\ Optimal\ Sum - Greedy\ Sum$$

The percent gap was calculated as:

$$Percent\ Gap = \frac{Quality\ Gap}{Brute\ Force\ Optimal\ Sum} \times 100$$

The greedy result was considered valid if its sum did not exceed the target value. The greedy result was considered optimal if its sum was equal to the brute force optimal sum.

7.3 Quality Testing Results

The quality test results were generated using Python and saved as:

`section7_quality_results.csv`

A summarized version of the results was also saved as:

`section7_quality_summary.csv`

Table 4: Quality testing summary for the greedy heuristic algorithm

n	Runs	Mean Quality Ratio	Minimum Quality Ratio	Mean Quality Gap	Mean Percent Gap	Greedy Optimal Rate
5	30	0.9792	0.8354	2.77	2.08	0.7333
6	30	0.9682	0.8649	4.40	3.18	0.4667
7	30	0.9743	0.9000	4.20	2.57	0.3333
8	30	0.9766	0.9333	4.67	2.34	0.2333
9	30	0.9743	0.8896	6.40	2.57	0.3000
10	30	0.9834	0.9327	4.37	1.66	0.2333
11	30	0.9808	0.9053	5.73	1.92	0.2000
12	30	0.9897	0.9485	3.40	1.03	0.2667
13	30	0.9895	0.9580	3.57	1.05	0.1333
14	30	0.9928	0.9687	2.53	0.72	0.3000
15	30	0.9926	0.9715	2.90	0.74	0.2333
16	30	0.9953	0.9823	1.90	0.47	0.4000
17	30	0.9943	0.9746	2.70	0.57	0.2667
18	30	0.9969	0.9834	1.50	0.31	0.5667

The mean quality ratio shows how close the greedy heuristic was to the brute force optimal answer on average. Across all tested input sizes, the mean quality ratio stayed very close to 1. This means that even when the greedy heuristic did not always find the exact optimal solution, it usually found a solution very close to the optimum.

The lowest mean quality ratio was 0.9682 for $n = 6$, which is still very close to 1. The highest mean quality ratio was 0.9969 for $n = 18$. This suggests that the greedy heuristic performed well in terms of solution quality across the tested input sizes.

The minimum quality ratio column shows the worst observed case for each input size. Even in the worst cases, the quality ratio remained reasonably high. The mean percent gap also stayed low, especially for larger input sizes. For example, for $n = 18$, the mean percent gap was only 0.31%.

The greedy optimal rate shows how often the greedy heuristic found the exact same sum as the brute force algorithm. This rate varied across input sizes. For example, the greedy heuristic matched the optimal solution in 73.33% of the cases for $n = 5$, but only 13.33% of the cases for $n = 13$. However, this does not mean the heuristic performed poorly, because the quality ratio stayed close to 1. In many cases, the greedy solution was not exactly optimal but was still very close to the optimal sum.

7.4 Visualization of Quality Ratio

The first quality chart shows the mean quality ratio of the greedy heuristic for different input sizes.

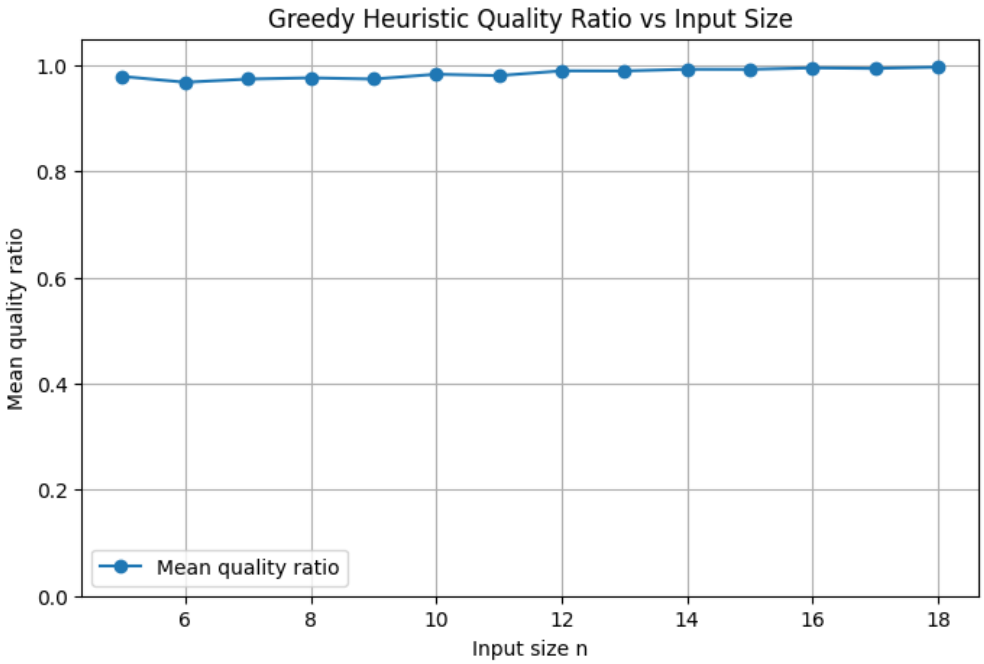

Figure 4: Mean quality ratio of greedy heuristic for different input sizes

This graph shows that the mean quality ratio remains close to 1 across all tested input sizes. This supports the conclusion that the greedy heuristic usually produces solutions very close to the brute force optimal solution. The line appears almost flat near the top of the graph because the quality ratios are consistently high.

7.5 Visualization of Optimal Match Rate

The second quality chart shows how often the greedy heuristic matched the brute force optimal solution.

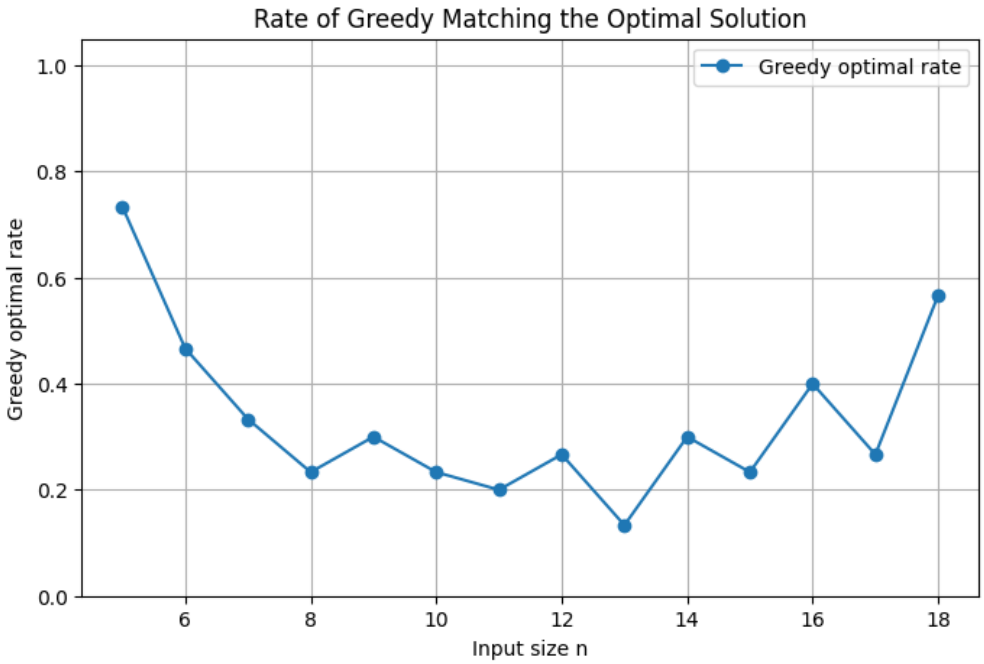

Figure 5: Rate of greedy heuristic matching the optimal solution

This graph shows the percentage of test cases where the greedy heuristic found the same subset sum as the brute force algorithm. The y-axis represents the greedy optimal rate. A value of 1.0 would mean that greedy matched the brute force optimal answer in 100% of the test cases for that input size. A value of 0.5 would mean that greedy matched the optimal answer in 50% of the test cases.

For each input size n, there were 30 random test cases. For example, when $n = 5$, the greedy optimal rate was about 0.7333. This means that for input size 5, the greedy heuristic matched the brute force optimal answer in about 73.33% of the cases, or around 22 out of 30 tests. However, when $n = 13$, the greedy optimal rate was about 0.1333. This means that for input size 13, the greedy heuristic matched the exact optimal answer in only about 13.33% of the cases, or around 4 out of 30 tests.

This graph changes noticeably across input sizes because exact matching depends heavily on the specific random numbers generated in each test case. In some cases, the greedy algorithm's choice of larger numbers happens to lead to the optimal subset. In other cases, choosing a large number early prevents the algorithm from selecting a better combination of smaller numbers.

However, this graph only measures exact optimal matches. It does not show how close the greedy solution was when it missed the optimal answer. This is why it should be interpreted together with the quality ratio graph. Even when the greedy optimal rate is low, the quality ratio can still be close to 1. This means the greedy heuristic may not always find the exact optimal subset, but it often finds a subset whose sum is very close to the brute force optimal sum.

7.6 Quality Discussion

The quality experiment compares the greedy heuristic against the brute force optimal result. This is important because the greedy heuristic is designed to be faster, but it does not guarantee optimality. By comparing greedy results with brute force results on smaller inputs, we can estimate how good the heuristic solutions are in practice.

The results show that the greedy heuristic produced high-quality solutions overall. The mean quality ratio stayed close to 1 for every tested input size. This means that the greedy heuristic usually found a subset sum very close to the optimal subset sum. The mean percent gap was also low, especially for larger input sizes.

However, the greedy heuristic did not always match the brute force optimal solution exactly. This is because the greedy algorithm selects large numbers first and does not reconsider earlier decisions. Choosing one large element early can prevent the algorithm from selecting a better combination of smaller elements later.

Overall, the quality experiment shows that the greedy heuristic has a strong tradeoff between speed and solution quality. It is much faster than brute force, as shown in Section 6, and it usually produces solutions that are close to optimal. Therefore, while the greedy heuristic is not guaranteed to find the best possible subset, it is still useful for larger Subset Sum instances where brute force is not practical.

8 Functional Testing

8.1 Purpose of Functional Testing

The purpose of this section is to test the correctness of the algorithm implementations. The correctness analysis in Section 3 explained why the algorithms are theoretically correct in their intended sense. However, even if an algorithm is correct in theory, coding errors can still be introduced during implementation. Therefore, the implementation must be tested to make sure that it behaves as expected.

For this project, functional testing was used to check both the brute force implementation and the greedy heuristic implementation. The brute force algorithm was tested by comparing its output to the expected optimal subset sum for known test cases. Since brute force is the exact algorithm, it should return the optimal feasible sum for each manually designed case.

The greedy heuristic was tested differently. Since the greedy heuristic is not guaranteed to find the optimal solution, it was not required to always match the brute force result. Instead, the main correctness requirement for the greedy heuristic implementation is that it returns a valid feasible subset. A greedy result is valid if its sum is less than or equal to the target value.

8.2 Testing Methods Used

The functional testing approach used both black-box and white-box testing methods.

Black-box testing is based on the requirements or specification of the program, without considering the source code or internal data. In this project, black-box testing was used by designing cases from the expected behavior of the Subset Sum problem. These included cases such as target equal to zero, a single element equal to the target, a single element greater than the target, duplicate values, no exact solution, multiple exact solutions, and all values larger than the target.

The test suite also used equivalence class testing. Equivalence class testing means grouping similar kinds of inputs and choosing representative tests from each group. For this project, the equivalence classes included exact-solution cases, no-exact-solution cases, duplicate-value cases, single-element cases, and cases where no nonempty subset can be selected.

White-box testing was also used. In white-box testing, the tester has access to the code and can design test cases to exercise important statements and decisions in the implementation. For this project, the white-box tests were designed to exercise the main branches of the brute force and greedy implementations. The tests cover cases where an element is added to the greedy subset, where an element is skipped because it would exceed the target, where an exact match is found, and where the returned solution is valid but not optimal.

The test suite supports statement coverage and decision coverage for the main parts of the implementation. Full path coverage was not claimed because the brute force algorithm generates many possible subsets, and covering every possible execution path would not be practical.

8.3 Functional Testing Results

The functional tests were generated and executed in Python. The results were saved as:

`section8_functional_testing_results.csv`

Because the full functional testing table is wide, it is split into two parts below.

Table 5: Functional testing design

Test	Test Name	Testing Method	Coverage Goal	Input List	Target
1	Target is zero	Black-box, boundary value	Empty subset / target zero case	[3, 5, 7]	0
2	Single element equals target	Black-box, equivalence class	Exact match with one element	[10]	10
3	Single element below target	Black-box, equivalence class	Feasible non-exact single element	[7]	10
4	Single element above target	Black-box, equivalence class	Single item skipped because it exceeds target	[12]	10
5	Exact subset exists	Black-box, normal case	Exact subset exists	[3, 5, 7, 10]	15
6	No exact subset exists	Black-box, equivalence class	No exact solution, best feasible sum returned	[2, 6, 8, 13]	11
7	Multiple exact solutions	Black-box, equivalence class	Multiple correct optimal subsets	[1, 3, 4, 5, 6]	9
8	Duplicate values	Black-box, equivalence class	Repeated values treated as separate items	[4, 4, 4, 6]	8
9	All values larger than target	Black-box, boundary value	All items skipped / empty subset returned	[20, 30, 40]	10
10	Greedy misses optimal	White-box, decision coverage	Greedy add/skip branches and non-optimal result	[10, 9, 6, 5]	11
11	Greedy exact match	White-box, statement coverage	Greedy reaches exact target branch	[9, 8, 6, 5]	14
12	Greedy skip branch	White-box, decision coverage	Greedy selects one item and skips later items	[15, 8, 7]	15

Table 6: Functional testing results

Test	Expected Brute Sum	Brute Force Sum	Brute Force Pass	Greedy Sum	Greedy Valid	Overall Pass
1	0	0	True	0	True	True
2	10	10	True	10	True	True
3	7	7	True	7	True	True
4	0	0	True	0	True	True
5	15	15	True	15	True	True
6	10	10	True	10	True	True
7	9	9	True	9	True	True
8	8	8	True	6	True	True
9	0	0	True	0	True	True
10	11	11	True	10	True	True
11	14	14	True	14	True	True
12	15	15	True	15	True	True

The table checks two different ideas. First, it checks whether the brute force algorithm returns the expected optimal value. Since brute force is the exact algorithm, it should match the expected optimal sum for each manually created test case. Second, it checks whether the greedy heuristic returns a valid result. The greedy result is valid if its sum is less than or equal to the target.

The overall pass column is marked true only when the brute force algorithm returns the expected optimal sum and the greedy heuristic returns a valid feasible sum. In all 12 test

cases, the brute force algorithm passed, the greedy heuristic returned a valid result, and the overall result was marked as true.

8.4 Discussion of Functional Tests

The functional tests were designed to check whether the implementation works correctly in different situations. The target-zero test checks whether the algorithms can correctly handle the case where the empty subset is the best solution. The single-element tests check whether the algorithms behave properly when there is only one item in the input list.

The duplicate-values test is important because the random sample generator can produce repeated numbers. Therefore, the implementation must treat repeated values as separate elements in the input list. In the duplicate-values test, the input list is $[4, 4, 4, 6]$ and the target is 8. The brute force algorithm correctly returns the optimal sum 8 by selecting two separate 4 values. The greedy heuristic returns 6, which is not optimal, but it is still valid because it does not exceed the target. This is acceptable because the greedy heuristic is not required to find the optimal solution during functional testing.

The all-values-larger-than-target test checks whether the algorithms correctly return an empty subset or sum 0 when no element can be selected. This is important because the selected subset must not exceed the target.

The no-exact-subset case checks whether the brute force algorithm still returns the best feasible sum even when no subset reaches the target exactly. For example, if the target is 11 and the input is $[2, 6, 8, 13]$, the expected optimal feasible sum is 10. The brute force algorithm returned 10, so it passed this test.

The greedy-misses-optimal test is also important. In this case, the input list is $[10, 9, 6, 5]$ and the target is 11. The brute force algorithm finds the optimal sum 11 using the subset $[6, 5]$. However, the greedy heuristic selects 10 first and then cannot add any other element without exceeding the target. Therefore, the greedy sum is 10. This test confirms that the implementation correctly demonstrates the limitation of the greedy heuristic while still returning a valid result.

The white-box tests also cover the main decision branches in the code. The greedy algorithm has a decision where it either adds an element if $current\_sum + x \leq T$ or skips the element if adding it would exceed the target. The test cases include both situations. The tests also include cases where the algorithms return an exact target match and cases where they return a non-exact feasible result.

Overall, the functional testing results support that the implementations behave as expected. The brute force implementation returned the correct optimal result for all known test cases. The greedy heuristic returned valid feasible results and never exceeded the target value. Therefore, the implementation appears to be correct for the tested cases.

9 Discussion

9.1 Discussion of Results

This project studied the Subset Sum problem using two different approaches: a brute force algorithm and a greedy heuristic algorithm. The brute force algorithm is an exact algorithm because it checks every possible subset and always finds the optimal subset sum that does not exceed the target value. However, the main disadvantage of brute force is that its running time grows exponentially as the input size increases. Because of this, brute force is only practical for small input sizes.

The greedy heuristic algorithm was used as a faster alternative. This algorithm sorts the input list in descending order and then adds each number if it does not make the current sum exceed the target. The heuristic is much faster than brute force because it does not check all possible subsets. Instead, it makes a sequence of local choices. This makes the algorithm scalable for much larger input sizes, but it also means that the algorithm is not guaranteed to find the optimal solution.

The experimental results showed this tradeoff clearly. In the performance testing section, the greedy heuristic ran efficiently even for input sizes much larger than what brute force could handle. In the quality testing section, the greedy heuristic did not always match the brute force optimal solution exactly. However, the mean quality ratio stayed close to 1, which means that the greedy heuristic usually found solutions close to the optimal solution.

9.2 Consistency Between Theory and Experiments

The theoretical analysis in Section 3 showed that the greedy heuristic has time complexity $\Theta(n \log n)$. This is because the algorithm first sorts the input list, which takes $\Theta(n \log n)$ time, and then scans the sorted list once, which takes $\Theta(n)$ time. Since sorting dominates the total running time, the final complexity is $\Theta(n \log n)$.

The experimental performance results were consistent with this theoretical analysis. As the input size increased, the mean running time of the greedy heuristic also increased. However, the increase was still manageable, even for large input sizes. The fitted running time model in Section 6 used $n \log_2(n)$ as the independent variable, which matches the theoretical complexity. The fitted line and log-log plot also supported the conclusion that the algorithm's practical running time follows the expected growth pattern.

There were no major inconsistencies between the theoretical analysis and the experimental analysis. The running times were not perfectly smooth, but this is normal in practical experiments. Python execution overhead, random input differences, memory behavior, and system load can all affect measured running time. To reduce the effect of random variation, repeated measurements were used along with 90% confidence intervals. Since all b/a values were below 0.1, the timing measurements were narrow enough according to the project

requirement.

9.3 Defects and Limitations of the Algorithm

The main defect of the greedy heuristic is that it does not always find the optimal solution. This is not a coding defect, but a limitation of the algorithmic strategy. The algorithm makes a local decision at each step by selecting the largest available value that can fit within the remaining target. However, a locally good choice is not always part of the globally optimal subset.

For example, consider the test case:

$$S = \{10, 9, 6, 5\}, \quad T = 11$$

The greedy heuristic selects 10 first because it is the largest value that does not exceed the target. After selecting 10, the algorithm cannot add 9, 6, or 5 because each would make the total exceed 11. Therefore, the greedy solution has sum 10. However, the optimal solution is $\{6, 5\}$, which has sum 11. This shows that the greedy heuristic can miss the optimal answer.

The quality testing results confirmed this limitation. The greedy optimal rate was not always high, meaning the heuristic did not always match the brute force optimal solution exactly. However, the quality ratio stayed close to 1, meaning that the heuristic usually produced solutions close to optimal even when it missed the exact optimal result.

9.4 Tradeoff Between Brute Force and Greedy

The main tradeoff in this project is between accuracy and speed. The brute force algorithm is accurate because it always finds the optimal solution, but it is slow for large inputs. The greedy heuristic is fast and scalable, but it is not always optimal.

For small input sizes, brute force is useful because it gives the exact answer and can be used to evaluate the quality of the heuristic. This is why brute force was used in the quality testing section. However, for large input sizes, brute force becomes impractical because the number of subsets grows as 2^n.

The greedy heuristic is more useful when the input size is large and a fast solution is needed. It can handle much larger instances because its running time grows approximately as $n \log n$. Although it does not guarantee the optimal solution, the experimental results showed that it usually produced high-quality solutions for the tested instances.

9.5 Functional Testing Discussion

The functional testing results showed that the implementations behaved correctly on the tested cases. The brute force algorithm returned the expected optimal sum for all manually

designed test cases. The greedy heuristic also returned valid feasible solutions in every test case, meaning its selected sum never exceeded the target value.

The functional tests included black-box test cases based on the problem requirements, such as target zero, single-element cases, duplicate values, no exact solution, multiple exact solutions, and all values larger than the target. The tests also included white-box cases that exercised important code decisions, such as adding an element, skipping an element, reaching an exact match, and showing a case where greedy misses the optimum.

These tests do not prove that the implementation is correct for every possible input, because exhaustive testing is not practical. However, they provide evidence that the implementation works correctly for important edge cases and representative input classes.

9.6 Final Conclusion

Overall, the theoretical and experimental results support each other. The brute force algorithm is correct and optimal, but it is not scalable because of its exponential running time. The greedy heuristic is not guaranteed to be optimal, but it is much faster and scales well to larger input sizes.

The performance testing showed that the greedy heuristic behaves consistently with its theoretical $\Theta(n \log n)$ time complexity. The quality testing showed that the heuristic often produces solutions close to the brute force optimal result. The functional testing showed that the implementation behaves correctly on important known cases and edge cases.

Therefore, the greedy heuristic is a practical approach for larger Subset Sum instances when a fast and reasonably good solution is acceptable. The brute force algorithm remains useful for small instances where the exact optimal solution is required or when it is needed as a benchmark for evaluating heuristic quality.

10 References

References

[1] Richard M. Karp. 1972. *Reducibility Among Combinatorial Problems*. In Complexity of Computer Computations, edited by R. E. Miller and J. W. Thatcher, Plenum Press, pp. 85–103.

[2] Michael R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms*. 3rd edition. MIT Press.